

```
<connectionStrings>
  <add name="ConnectionString" connectionString="Data Source=(localdb)\MSSQLLocalDB;
    Initial Catalog=EzBiology;Integrated Security=True" providerName="System.Data.SqlClient" />
</connectionStrings>
```

The screenshot above displays the connection string setting in the web.config file, which is utilized to connect EzBiology with its dedicated database. This connection string is used throughout the system – It connects to the local Microsoft SQL server and connects to the EzBiology database.

Lofin

```
public partial class Login : System.Web.UI.Page
{
    private SqlConnection conn = new
        SqlConnection(ConfigurationManager.ConnectionStrings["ConnectionString"].ConnectionString);
    private string passwordQuery = "SELECT Password FROM Users WHERE Username=@username";
    private string roleQuery = "SELECT Role FROM Users WHERE Username=@username";
    private string userIDQuery = "SELECT UserID FROM Users WHERE Username=@username";
    private string activeQuery = "SELECT IsActive FROM Users WHERE Username=@username";
    private string deletedQuery = "SELECT IsDeleted FROM Users WHERE Username=@username";
```

The screenshot above displays the SqlConnection variable that uses the established connection string and several SQL queries in string form used in the Login page. The SQL queries perform different operations in the Users table, which are as follows:

- passwordQuery: Retrieve the hashed Password data of the entered username.
- roleQuery: Retrieve Role data of the entered username.
- userIDQuery: Retrieve UserID data of the entered username.
- activeQuery: Retrieve isActive data of the entered username, which is the active status of an user.
- deletedQuery: Retrieve IsDeleted data of the entered username, which is the deletion status of an user.

```
protected void btnSubmit_Click(object sender, EventArgs e)
{
    if (!Page.IsValid) return;

    string username = txtUsername.Text.Trim();
    string password = txtPassword.Text;

    conn.Open();
```

```

SqlCommand cmdDeletedCheck = new SqlCommand(deletedQuery, conn);
cmdDeletedCheck.Parameters.AddWithValue("@username", username);
bool isDeleted = Convert.ToBoolean(cmdDeletedCheck.ExecuteScalar());
if (isDeleted)
{
    errorMessage.Text = "Your account has been deleted.";
    errorMessage.Visible = true;
    return;
}

```

The screenshot displays the SqlCommand variable cmdDeletedCheck, which utilizes the deletedQuery string and conn to verify the deletion status of the user-entered username. After the parameter value "@username" is added with the username string value, which is the value retrieved from the username textbox, the command is run through the ExecuteScalar method and its returned value is converted to boolean. This boolean value is then stored in a boolean variable and is utilized in an if statement. If the boolean variable contains true, which denotes that a user is deleted, the system will display an error message and exit from the login method.

```

SqlCommand cmd = new SqlCommand(passwordQuery, conn);
cmd.Parameters.AddWithValue("@username", username);
object pwdResult = cmd.ExecuteScalar();
if (pwdResult != null)
{
    string dbPassword = pwdResult.ToString();
    bool isPassword = BCrypt.Net.BCrypt.Verify(password, dbPassword);
    if (isPassword)
    {

```

The screenshot displays a SqlCommand variable called cmd, which implements the passwordQuery string and conn. After the parameter is added with the retrieved username string value from its dedicated textbox, it's executed through ExecuteScalar and the returned hashed password value from said execution is stored in an object variable called pwdResult. The login process will only continue if pwdResult isn't null, since a null object variable means that the username doesn't exist. If pwdResult isn't null, the hashed password object is converted to string and is compared with the user-entered password through BCrypt's verify method. This function hashes the user-entered password with a hashing algorithm, compares it to the database-retrieved password, and returns a boolean value based on the comparison result – true if they're the same, false if the opposite. If the entered password is a match, the system will proceed to with the login method execution

```
//check if user is active
SqlCommand cmdActiveCheck = new SqlCommand(activeQuery, conn);
cmdActiveCheck.Parameters.AddWithValue("@username", username);
bool isActive = Convert.ToBoolean(cmdActiveCheck.ExecuteScalar());
if (!isActive)
{
    errorMessage.Text = "Your account is inactive.";
    errorMessage.Visible = true;
    return;
}

//get role
SqlCommand cmd2 = new SqlCommand(roleQuery, conn);
cmd2.Parameters.AddWithValue("@username", username);
string roleResult = cmd2.ExecuteScalar().ToString();

//get userID
SqlCommand cmd3 = new SqlCommand(userIDQuery, conn);
cmd3.Parameters.AddWithValue("@username", username);
string userIDResult = cmd3.ExecuteScalar().ToString();

Session["Role"] = roleResult;
Session["Username"] = username;
Session["UserID"] = userIDResult;
```

The screenshot above displays the continuation of the Login method. After the password is verified, the system will retrieve the user's active status through the SqlCommand variable cmdActiveCheck. After adding it with the entered-username, cmdActiveCheck is executed with ExecuteScalar and its returned value is stored in a boolean variable called isActive after it's converted to boolean. The system will stop the login process if isActive is false and display an error message to the user. If the user is active, the system will get the user's Role and UserID values and store them in string variables. Then, the user's role, username, and user ID are assigned to separate Session variables.

```

        switch (roleResult)
        {
            case "Admin":
                Response.Redirect("AdminDashboard.aspx");
                break;
            case "Student":
                Response.Redirect("StudentDashboard.aspx");
                break;
            case "Teacher":
                Response.Redirect("Home.aspx");
                break;
        }
    }
    errorMessage.Visible = true;
}

```

The screenshot above displays the last section of the Login method. A switch statement is utilized on roleResult, which stores the user's role data. The statement redirects the user to different pages according to their role:

- Admins: Redirected to AdminDashboard.aspx
- Student: Redirected to StudentDashboard.aspx
- Teacher: Redirected to Home.aspx (The Teacher Dashboard Page).

Additionally, the set error message to visible is put outside of the password if statement, so it displays the appropriate error message if the user-entered password fails the verification.

Sign up

```

public partial class Signup : System.Web.UI.Page
{
    private SqlConnection conn =
        new SqlConnection(ConfigurationManager.ConnectionStrings["ConnectionString"].ConnectionString);
    private string insertQuery = "INSERT INTO Users (Username, Password, Role, " +
        "FullName, IsActive, IsDeleted)" +
        "VALUES (@username, @password, @role, @name, @active, @deleted)";
    private string checkQuery = "SELECT * FROM Users WHERE Username = @username";
}

```

The screenshot above displays the Signup class, which is utilized in the user signup page. The class first creates a SqlConnection variable called conn and two SQL queries stored as string in string variables. The first string variable called insertQuery is an INSERT query Used to insert new users into the Users table. The second variable called checkQuery is a SELECT query used to retrieve a user's complete credential if said user has the same Username value as the added parameter value.

0 references

```
protected void btnSubmit_Click(object sender, EventArgs e)
{
    if (!Page.IsValid) return;

    string fullName = txtFullName.Text.Trim();
    string role = rolePick.Text;
    string username = txtUsername.Text.Trim();
    string password = txtPassword.Text;

    conn.Open();
    SqlCommand cmd = new SqlCommand(checkQuery, conn);
    cmd.Parameters.AddWithValue("@username", username);
    object found = cmd.ExecuteScalar();

    if (found == null)
    {
        SqlCommand cmd2 = new SqlCommand(insertQuery, conn);
        cmd2.Parameters.AddWithValue("@username", username);
        cmd2.Parameters.AddWithValue("@password", BCrypt.Net.BCrypt.HashPassword(password));
        cmd2.Parameters.AddWithValue("@role", role);
        cmd2.Parameters.AddWithValue("@name", fullName);
        cmd2.Parameters.AddWithValue("@active", 1);
        cmd2.Parameters.AddWithValue("@deleted", 0);
        cmd2.ExecuteNonQuery();
        Response.Redirect("Login.aspx");
    }
    else
    {
        errMsg.Visible = true;
    }
}
```

The screenshot above displays the Signup code in the btnSubmit_Click method, which is called if the submit button is clicked. The method starts by first verifying that all of the validation controls are validated through the Page's IsValid method. The system will only proceed if all validations are successful. Then, the entered user credentials are retrieved from textboxes or dropdowns and put into string variables. After all of the user-entered information is retrieved, the system will open a connection to the database and create a new SqlCommand variable that implements the checkQuery string. This variable called cmd is added with entered username value and is executed with ExecuteScalar. The returned object is then compared to null in an if statement to verify if a user with the same username already exists. The system will display an error message if the object isn't null. If a user with the same username doesn't exist, the system will create another SqlCommand variable called cmd2. Cmd2 utilizes the insertQuery string, and all of its parameters except for "@password" are added with the string variables which contain the entered user information. The parameter "@password" is instead added with the hashed value of password which is received through BCrypt's HashPassword function. After all of the parameters are added with value, the SqlCommand is executed and the user is redirected to the login page.

Css class

```
/* — Auth Nav (minimal: logo | divider | single link) — */
.auth-nav {
  background: var(--green-nav);
  height: 68px;
  display: flex;
  align-items: center;
  padding: 0 36px;
  position: sticky;
  top: 0;
  z-index: 200;
}

.auth-nav-divider {
  width: 1px;
  height: 68px;
  background: rgba(255,255,255,0.4);
  margin-left: auto;
  margin-right: 36px;
}

.auth-nav-link {
  font-family: 'Instrument Sans', sans-serif;
  font-size: 22px;
  font-weight: 500;
  color: var(--white);
  text-decoration: none;
  white-space: nowrap;
}
```

The screenshot above displays the CSS classes utilized in the navigation bars of user authentication pages. The first CSS class, `.auth-nav`, is used to style the navigation bar shown at the top of the authentication pages. It applies a green background colour using `var(--green-nav)`, which gives the pages a consistent EzBiology theme. The height value of 68px sets the navigation bar height into a set vertical size, while a flex display and a center `align-items` align the navigation content in the middle vertically. The top and right padding values of 0px and 36px respectively add horizontal spacing on the left and right sides of the navigation bar. The sticky position and 0 top value allow the navigation bar to remain fixed at the top of the page when the user scrolls. The 200 z-index value ensures that the navigation bar appears above other page elements. The `.auth-nav-divider` class creates a thin vertical divider inside the navigation area. Its width is set to 1px and height is set to 68px, which makes it a narrow vertical line. The `background: rgba(255, 255, 255, 0.4)` gives it a semi-transparent white colour, which separates the logo area from the navigation link while still matching the overall design. The `.auth-nav-link` class is used for the navigation link text. It uses the Instrument Sans font, a 22px font size, and a medium font weight to make the link readable and professional. The color set to

`var(--white)` property makes the link white, and `text-decoration: none` removes the default underline. The `nowrap` `white-space` property prevents the link text from wrapping onto a new line.

```
/* — Auth Page Layout — */
.auth-page {
  display: grid;
  grid-template-columns: 46% 54%;
  min-height: calc(100vh - 68px);
  background: var(--white);
}
```

The screenshot above displays the `.auth-page` class, which defines the layout of authentication pages. It uses a grid display and `grid-template-columns` that are set to 46% and 54% respectively. This effectively divides the page into two main sections – The first section is a left hero and the second section is a right form. The left side occupies 46% of the page width and the right side takes 54%. The `min-height: calc(100vh - 68px)` property ensures that the authentication page fills the available screen height while subtracting the height of the navigation bar. This helps the page maintain a full-screen appearance without overlapping the navigation bar. The `background: var(--white)` property sets the authentication page background color as white.

```
/* — Left Panel (DNA hero) — */
.auth-hero {
  position: relative;
  overflow: hidden;
  display: flex;
  flex-direction: column;
  justify-content: flex-end;
  padding: 52px 48px;
}

.auth-hero-bg {
  position: absolute;
  inset: 0;
  z-index: 0;
}

.auth-hero-bg img {
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.auth-hero-overlay {
  position: absolute;
  inset: 0;
  background: var(--green-overlay);
}

.auth-hero-text {
  position: relative;
  z-index: 1;
}

.auth-hero-text h1 {
  font-family: 'Inclusive Sans', sans-serif;
  font-size: 72px;
  font-weight: 700;
  color: var(--white);
  line-height: 1.1;
  margin-bottom: 20px;
}

.auth-hero-text p {
  font-family: 'Instrument Sans', sans-serif;
  font-size: 26px;
  font-weight: 600;
  color: var(--white);
  line-height: 1.35;
}
```


The screenshot above displays the CSS classes used in the left panel of the authentication page. It utilizes a relative position so that child elements, such as the background image and overlay, can be positioned inside it. The hidden overflow property prevents any image or overlay content from extending outside the hero panel. The class also uses flex display, a column flex-direction, and a flex-end justify-content property to position the hero text near the bottom of the section. The top and right padding of 52px and 48px give the text enough spacing from the edges. The .auth-hero-bg class positions the background image container. It uses an absolute position, and sets both inset and z-index to 0. This setting makes the background cover the entire hero section while staying behind the text content.

The .auth-hero-bg img selector styles the image inside the hero background. The 100% width and height properties make the image fill the whole container. The cover object-fit property ensures that the image covers the available area and prevents image distortion, even if the image aspect ratio is different from the container. The .auth-hero-overlay class adds an overlay on top of the background image. It also uses an absolute position and 0 inset to cover the entire image. The background: var(--green-overlay) property applies a green transparent layer, making the image darker and more consistent with the EzBiology colour theme.

The .auth-hero-text class puts the text above the background image and overlay by using a relative position and z-index value of 1. This ensures that the text appears on top of the image section clearly. The .auth-hero-text h1 selector styles the main hero heading. It uses the Inclusive Sans font, a large 72px font size, and a bold 700 font weight to create a strong visual title. The 1.1 line height keeps the heading compact, and a margin-bottom value of 20px creates spacing between the heading and the paragraph below it. The .auth-hero-text p selector styles the supporting paragraph below the heading. It uses the Instrument Sans font, a 26px font size, and a 600 font weight. The 1.35 line-height improves readability by giving enough spacing between lines.

```

/* — Right Panel (form) — */
.auth-form-panel {
  display: flex;
  flex-direction: column;
  justify-content: center;
  padding: 48px 72px;
  background: var(--white);
}

.auth-field {
  margin-bottom: 24px;
}

.auth-field label {
  display: block;
  font-family: 'Instrument Sans', sans-serif;
  font-size: 18px;
  font-weight: 400;
  color: var(--black);
  margin-bottom: 8px;
}

.auth-field input {
  font-family: 'Instrument Sans', sans-serif;
  font-size: 18px;
  color: var(--grey-ph);
  width: 100%;
  border: 1px solid var(--black);
  border-radius: 10px;
  padding: 14px 18px;
  box-shadow: 0 4px 4px rgba(0,0,0,0.15);
  outline: none;
  background: var(--white);
  transition: border-color .15s;
}

.auth-field input:focus {
  border-color: var(--green-nav);
  color: var(--black);
}

```

The screenshot above displays the CSS classes used in the right panel of the authentication page. The `.auth-form-panel` class is used to style the right section of the authentication page where the login or signup form is displayed. It uses a flex display, a column flex-direction, and a center justify-content property to center the form content vertically. The 48px top padding and 72px right padding give the form enough space from the edges of the panel. The background property set to `var(--white)` ensures the form area is clear and easily readable. The `.auth-field` class is used as a wrapper for each form input group. It applies a 24px

margin-bottom to create a consistent spacing between each field. This prevents the form controls from being too close to one another.

The `.auth-field` label selector styles the label above each input field. It uses a block display so the label appears on its own line above the input. The font is set to Instrument Sans, with an 18px size and 400 font weight. The color property set to `var(--black)` makes the label easy to read, and 8px margin-bottom creates spacing between the label and its input field. The `.auth-field` input selector styles the input boxes used in the form. It sets the font to Instrument Sans, the font size to 18px, and the text colour to `var(--grey-ph)`. The 100% width property allows the input to stretch across the available form width. The 1px border solid `var(--black)` property creates a visible border around the input, while 10px border-radius gives the input rounded corners. The 14px top margin and 18px right margin add internal spacing, which makes the input more comfortable to use. The box-shadow property adds a subtle shadow, giving the input a slight depth effect. The outline set to none removes the default browser outline, and a transition of border-color .15s makes the border colour change smoothly when the input is focused.

The `.auth-field input:focus` selector changes the input border colour when the user clicks or tabs into the field. The `border-color: var(--green-nav)` property highlights the active input using the same green colour as the navigation bar. This provides visual feedback and helps users identify which field they are currently editing.

```
.auth-select {
  font-family: 'Instrument Sans', sans-serif;
  font-size: 18px;
  color: #6e6e6e;
  width: 50%;
  border: 1px solid #000;
  border-radius: 10px;
  padding: 14px 18px;
  box-shadow: 0 4px 4px rgba(0,0,0,0.15);
  outline: none;
  background: #ffffff;
  -webkit-appearance: none;
  -moz-appearance: none;
  appearance: none;
  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
  background-repeat: no-repeat;
  background-position: right 16px center;
  padding-right: 40px; /* make room for the arrow */
  cursor: pointer;
}

.auth-select:focus {
  border-color: var(--green-nav);
  color: var(--black);
}
```

The screenshot above displays the `.auth-select` class and `.auth-select: focus` selector, a part of the CSS classes used in the right panel. The `.auth-select` class is used to style dropdown lists in the authentication form, such as the role selection or gender selection field. It makes the dropdown visually consistent with the other input controls by applying the Instrument Sans font, an 18px font size, a white background, rounded corners, a black border, and internal padding. The 50% width property makes the dropdown take up half of its container, while box-shadow gives it a subtle depth effect. The outline set to none effectively removes the default browser outline so that custom focus styling can be used instead. The browser's default dropdown arrow is removed through these 3 settings which are:

- `-webkit-appearance` set to none
- `-moz-appearance` set to none
- `appearance` set to none

After removing the default arrow, `background-image` adds a custom SVG arrow directly through CSS. `background-repeat: no-repeat` prevents the arrow from repeating, while a `right 16px center` `background-position` places it neatly on the right side of the dropdown. The `40px padding-right` property gives the arrow enough space so it does not overlap with the selected text. The `.auth-select:focus` selector changes the border colour to `var(--green-nav)` and the text colour to `var(--black)` when the dropdown is selected. This provides clear visual feedback to the user and keeps the interaction style consistent.

```

/* — Auth Buttons — */
.auth-btns {
  display: flex;
  gap: 20px;
  margin-top: 12px;
}

.auth-btns .btn {
  font-size: 18px;
  padding: 14px 40px;
  border-radius: 15px;
}

/* — Validation / Message — */
.auth-message {
  display: block;
  font-family: 'Instrument Sans', sans-serif;
  font-size: 15px;
  font-weight: 600;
  margin-bottom: 16px;
  min-height: 20px;
}

.auth-validator {
  font-family: 'Instrument Sans', sans-serif;
  font-size: 13px;
  color: var(--red-btn);
  display: block;
  margin-top: 4px;
}

```

The screenshot above displays the CSS classes used for the button elements in the authentication page. The `.auth-btns` class is used as the container for the form buttons. It applies a flex display to make the buttons appear in one row. The 20px gap property creates spacing between the buttons, while a 12px margin-top separates the button row from the form fields. The `.auth-btns .btn` selector styles the buttons inside the authentication form. It sets the font size to 18px, applies a top and right padding of 14px and 40px respectively, and uses 15 px border-radius to create large rounded buttons. These properties make the buttons more visually prominent and easier to click.

The `.auth-message` class is used for displaying general messages to the user, such as login errors or account status messages. It uses a block display so the message appears on its own line. The font is set to Instrument Sans, with a 15px size and 600 font weight, making the message noticeable. The 16px margin-bottom separates it from nearby fields, while a 20px min-height reserves space for the message even when no text is shown. This helps prevent the form layout from shifting suddenly when a message appears. The `.auth-validator` class is used for ASP.NET validation messages. It uses the same Instrument Sans font and sets the font size

to 13px. The color set to `var(--red-btn)` makes validation errors appear in red, which clearly indicates to the user that their input needs correction. The `block display` property makes each validation message appear below its related field, and a `4px margin-top` creates a small gap between the input field and the validation text.